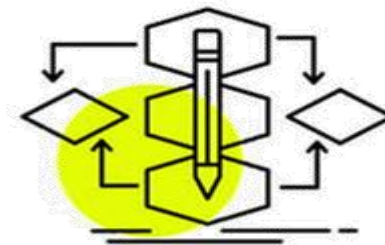


Partie 1: Raisonnement par Algorithme

CHAPITRE 4

Les Tableaux



Module : Informatique 1 : Algorithmique 1/Python

Pr. Mariya OUAISSA
E-mail : m.ouaissa@uca.ac.ma
Département : Informatique FSSM

Année Universitaire : 2023/2024

1. Introduction - Tableau



Imaginons que dans un programme, nous ayons besoin simultanément de 12 valeurs (par exemple, des notes pour calculer une moyenne).

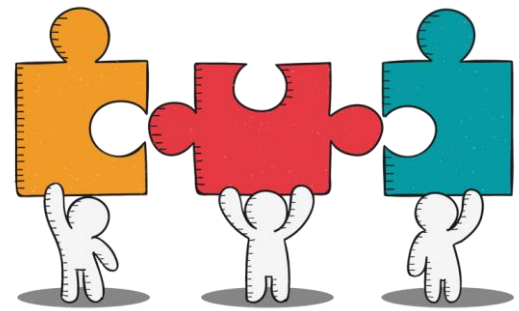
Evidemment, la seule solution dont nous disposons à l'heure actuelle consiste à déclarer douze variables, appelées par exemple Notea, Noteb, Notec, etc.

Bien sûr, on peut opter pour une notation un peu simplifiée, par exemple N1, N2, N3, etc. Mais cela ne change pas fondamentalement notre problème, car arrivé au calcul, et après une succession de douze instructions « Lire » distinctes, cela donnera obligatoirement une atrocité du genre :

$$\text{Moy} \leftarrow (N1+N2+N3+N4+N5+N6+N7+N8+N9+N10+N11+N12)/12$$

Solution ?

- Et si on a une centaine de valeur à calculer ??
- si en plus on est dans une situation on l'on ne peut pas savoir d'avance combien il y aura de valeurs à traiter, là on est carrément dans le complexe.
- C'est pourquoi **la programmation nous permet de rassembler toutes ces variables en une seule**, au sein de laquelle chaque valeur sera désignée par un numéro.



Enfin une définition du **Tableau**

Un ensemble de valeurs portant le même nom de variable et repérées par un nombre, s'appelle un **tableau**, ou encore une variable indicée.

Le nombre qui, au sein d'un tableau, sert à repérer chaque valeur s'appelle l'indice.

Chaque fois que l'on doit désigner un élément du tableau, on fait figurer le nom du tableau, suivi de l'indice de l'élément.

Syntaxe

Tableau nom_Tableau[Nb_Element] : **Type**

Tab : tableau [nb_Element] de entier

Exemple des notes

Certains langages réclament le nombre d'éléments, d'autre le plus grand indice...

- Dans notre exemple, nous créerons donc un tableau appelé Note. Chaque note individuelle (chaque emplacement du tableau Note) sera donc désignée Note[0], Note[1], etc. Eh oui, attention, les indices des tableaux, par convention, commencent *généralement* à 0, et non à 1.

Note : Tableau [12] de entier

- les "cases" sont numérotées à partir de zéro, autrement dit que le plus petit indice est zéro.
- lors de la déclaration d'un tableau, on précise la plus grande valeur de l'indice (différente, donc, du nombre de cases du tableau, puisque si on veut **12** emplacements, le plus grand indice sera **11**). Au début, ça déroute, mais vous verrez, avec le temps, on se fait à tout, même au pire.

Note	10.5	15	16	13.5	11	19	18.5	14.5	11	12	12	9
	0	1	2	3	4	5	6	7	8	9	10	11

← indice

A retenir

si l'on est dans un langage où les indices commencent à zéro, il faut en tenir compte à la déclaration :

Tableau Note[14] : réel

créera un tableau de 14 éléments, le plus petit indice étant 0 et le plus grand 13.

LE GAG DE LA JOURNEE

Il consiste à confondre, dans sa tête et / ou dans un algorithme, l'**indice** d'un élément d'un tableau avec le **contenu** de cet élément. La troisième maison de la rue n'a pas forcément trois habitants, et la vingtième vingt habitants. En notation algorithmique, il n'y a aucun rapport entre i et $\text{truc}[i]$.

2. Tableaux à une dimension – Déclaration

La déclaration d'un tableau s'effectue en précisant 3 éléments fondamentaux :

- **Nom_du_tableau** : Identificateur respectant les règles classiques d'identificateur d'une donnée;
- **La taille**: Le nombre de ses valeurs. Ce nombre se nomme également la *taille maximale* du tableau;
- **Le type** : correspond au type des valeurs qu'il contiendra;

2. Tableaux à une dimension – Déclaration

Syntaxe :

Variable ***tableau*** NomDuTableau[taille] : Type;

Exemple :

```
Algorithme exemple;  
Variable      tableau Notes [40] : réel;  
Début  
    ...  
Fin.
```


2. Tableaux à une dimension – Représentation

- Un tableau peut être vu comme un ensemble de « cases » où chaque case stocke une valeur. Soit la définition suivante :

Variable *tableau* Notes[8] : réel;

- Cette déclaration crée un tableau appelé *notes* qui stocke au **maximum** huit *réels* dans huit cases différentes. Aucune de ces cases n'est initialisée, ce qui est indiqué par un « ? ».
- Soit *i* un indice (*i* est donc de type entier puisqu'il s'agit d'un numéro de case). La valeur stockée dans la case d'indice *i* du tableau *notes* se nomme *notes[i]*

<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>
?	?	?	?	?	?	?	?
Notes[0]	Notes[1]	Notes[2]	Notes[3]	Notes[4]	Notes[5]	Notes[6]	Notes[7]

2. Tableaux à une dimension – Remarques

- La taille correspond au nombre maximum de cases composant le tableau, elle **doit être une constante numérique**;
- On peut définir des tableaux de tous types : tableaux d'entiers, de réels, de caractères, de booléens ou de chaînes de caractères, ...
- L'accès à un élément du tableau se fait au moyen de l'indice;
- Selon les langages, le **premier indice** du tableau est **soit 0, soit 1. Le plus souvent c'est 0** (*c'est ce qu'on va adopter en pseudo-code*). Dans ce cas, `notes[i]` désigne l'élément $i+1$ du tableau `notes`;

2. Tableaux à une dimension – Remarques

- Il ne faut pas confondre la valeur notée entre crochets lors de la définition du tableau (**la taille maximale**) et la valeur notée entre crochets lors des instructions (**l'indice**).
- Toute expression qui donne une **valeur entière** peut jouer le rôle **d'indice** lors de l'accès aux valeurs stockées dans un tableau.
- La notation *notes[i]* est équivalente à une variable et peut être utilisée comme telle dans un algorithme. Elle est considérée comme ayant le type des éléments stockés dans le tableau.
- Les données stockées dans un tableau peuvent être manipulées en utilisant les boucles.

2. Tableaux à une dimension – Affectation, lecture et écriture

Les instructions de *lecture*, *écriture* et *affectation* s'appliquent aux tableaux comme aux variables.

Exemples :

➤ `notes[0] ← 13;`

Cette instruction affecte 13 au 1^{er} élément du tableau notes.

➤ `note ← notes[0];`

La variable note prend la valeur du premier élément du tableau notes (13 selon l'instruction précédente).

➤ `Lire(notes[1]);`

Cette instruction permet de lire une valeur à partir du clavier et de l'assigner au 2^{ème} élément du tableau notes.

➤ `Ecrire(notes[2]);`

Cette instruction permet d'afficher la valeur du 3^{ème} élément du tableau notes.

2. Tableaux à une dimension – Relation entre tableaux et boucles

- Les boucles sont extrêmement utiles pour les algorithmes associés aux tableaux.
- Pour parcourir les éléments du tableau selon l'ordre croissant (ou décroissant) des indices, on utilise des boucles.
- Le traitement de chacun des éléments étant souvent le même, seule la valeur de l'indice est amenée à changer.
- Une boucle est donc parfaitement adaptée à ce genre de traitements.

Exercices:



Exercice 1

Ecrire un algorithme qui déclare et remplit un tableau de 7 valeurs numériques en les mettant toutes à zéro.

Exercice 2

Ecrire un algorithme qui déclare et remplit un tableau contenant les six voyelles de l'alphabet latin.

Exercice 3

Ecrire un algorithme qui déclare un tableau de 9 notes, dont on fait ensuite saisir les valeurs par l'utilisateur.

Exercice 4

Que produit l'algorithme suivant ? Peut-on simplifier cet algorithme avec le même résultat ?

Tableau Nb[6] : Entier

Variable i : Entier

Début

Pour i $\leftarrow$ 0 à 5

Nb[i] $\leftarrow$ i * i

fin Pour

Pour i $\leftarrow$ 0 à 5

Ecrire Nb[i]

fin Pour

Fin

Correction

```
Tableau Truc[7] en Numérique  
Variable i en Numérique  
Debut  
Pour i ← 0 à 6  
    Truc[i] ← 0  
i Suivant  
Fin
```

1

```
Tableau Truc[6] en Caractère  
Debut  
Truc[0] ← "a"  
Truc[1] ← "e"  
Truc[2] ← "i"  
Truc[3] ← "o"  
Truc[4] ← "u"  
Truc[5] ← "y"  
Fin
```

2

```
Tableau Notes[9] en Numérique  
Variable i en Numérique  
Pour i ← 0 à 8  
    Ecrire "Entrez la note numéro ", i + 1  
    Lire Notes[i]  
i Suivant  
Fin
```

3

4

Cet algorithme remplit un tableau avec six valeurs : 0, 1, 4, 9, 16, 25.
Il les écrit ensuite à l'écran. Simplification :

```
Tableau Nb[6] en Numérique  
Variable i en Numérique  
Début  
Pour i ← 0 à 5  
    Nb[i] ← i * i  
    Ecrire Nb[i]  
i Suivant  
Fin
```

3. Tableaux à deux dimensions

- Les langages de programmation permettent de déclarer des tableaux dans lesquels les valeurs sont repérées par deux indices.
- Ceci est utile par exemple pour représenter des matrices.
- En pseudo-code, un tableau à deux dimensions se déclare ainsi :

Variable *tableau* nomDuTableau[taille_n][taille_m] : *Type*;

Exemple :

- Une matrice A de 3 lignes et 4 colonnes dont les éléments sont entiers:

Variable *tableau* A[3][4] : *entier*;

- ***A[i][j]*** permet d'accéder à l'élément de la matrice qui se trouve à l'intersection de la ligne ***i+1*** et de la colonne ***j+1***;
- Les tableaux peuvent avoir *n dimensions*.

3. Tableaux à deux dimensions

- La matrice **A** dans la déclaration suivante :

Variable *tableau* A[3][5] : entiers;

peut être explicitée comme suit : les éléments sont rangés dans un tableau à deux entrées.

	0	1	2	3	4
0	15	45	17	36	98
1	52	47	56	58	23
2	15	46	87	95	66

- Ce tableau a 3 lignes et 5 colonnes. Les éléments du tableau sont repérés par leur numéro de ligne et leur numéro de colonne. Par exemple A[1][2] vaut 56.

3. Tableaux à deux dimensions – Exemple : lecture d'une matrice

```
Algorithme lectureMatrice;  
Variable          i, j, m, n : entier;  
                  tableau A[100][100] : réel;  
  
Début  
Ecrire("Entrer le nombre de lignes et le nombre de colonnes :");  
lire(m, n);  
Pour i de 0 à m-1 faire  
    Ecrire ("Saisie de la ligne ", i+1);  
    Pour j de 0 à n-1 faire  
        Ecrire ("Entrer l'élément : A[" , i+1, "][" , j+1, "]");  
        Lire (A[i][j]);  
    FinPour  
FinPour  
Fin
```

3. Tableaux à deux dimensions – Exemple : affichage d'une matrice

```
Algorithme affichageMatrice;  
Variables      i, j, m, n : Entier;  
                tableau A[100][100]: Réel;  
  
Début  
    Ecrire("Entrer le nombre de lignes et le nombre de colonnes : ");  
    Lire(m, n);  
    //Lecture de la matrice  
    ...  
    //Affichage de la matrice  
    Pour i de 0 à m-1 faire  
        Pour j de 0 à n-1 faire  
            Ecrire ("A[" ,i, " ] [" ,j, "]=", A[i][j]);  
        FinPour  
    FinPour  
Fin
```

Exercices:



Exercice 5

Ecrire un algorithme qui déclare et remplit un tableau de taille (6,13) en mettant toutes ces valeurs à zéro.

```
Algorithme tab2d;  
Variables    tableau M[6][13] : entier;  
             i, j : entier;  
  
Début  
    Pour i de 0 à 5 faire  
        Pour j de 0 à 12 faire  
            M[i][j] ← 0;  
        FinPour  
    FinPour  
Fin
```

4

4. Tri d'un tableau

- Le tri consiste à ordonner les éléments du tableau dans l'ordre croissant ou décroissant.
- Un tableau **T** est dit « **trié en ordre croissant** » si tous les éléments consécutifs du tableau vérifient :
$$T[i-1] \leq T[i]$$
- D'où la définition:
 - Un tableau vide ($n=0$) est ordonné (trié),
 - Un tableau contenant un seul élément $n=1$ est ordonné,
- Il existe plusieurs algorithmes connus pour trier les éléments d'un tableau
 - Le tri par sélection; (*Cas traité*)
 - Le tri à bulles; (*Cas traité*)
 - Le tri rapide;
 - ...

4. Tri d'un tableau – Tri par sélection

Principe :

- A l'étape i , on sélectionne le plus petit élément parmi les $(n-i+1)$ éléments du tableau les plus à droite. On l'échange ensuite avec l'élément i du tableau.

Exemple :

9	4	1	7	3
---	---	---	---	---

- Étape 1: on cherche le plus petit parmi les 5 éléments du tableau. On l'identifie en troisième position, et on l'échange alors avec le premier élément :

1	4	9	7	3
---	---	---	---	---

- Étape 2: on cherche le plus petit élément, mais cette fois à partir du deuxième élément. On le trouve en dernière position, on l'échange avec le deuxième:

- Étape 3:

1	3	9	7	4
---	---	---	---	---

1	3	4	7	9
---	---	---	---	---

4. Tri d'un tableau – Tri par sélection

Algorithme (Pseudo-code) :

```
...  
Pour i de 0 à N-1 faire                                //N est le nombre d'éléments du tableau T  
    i_ppe ← i;                                           //i_ppe est l'indice de plus petit élément  
    Pour j de i + 1 à N faire  
        Si T[j] < T[i_ppe] alors  
            i_ppe ← j;  
        Finsi  
    FinPour  
    temp ← T[i_ppe]; //temp est une variable temporaire pour la permutation  
    T[i_ppe] ← T[i];  
    T[i] ← temp;  
FinPour  
...
```

4. Tri d'un tableau – Tri à bulles

Principe:

- Le tri à bulles est un algorithme de tri qui consiste à faire remonter progressivement les plus grands éléments d'un tableau (comme des bulles remontent à la surface d'un verre de soda).
- Le tri à bulles est une succession d'étapes. Une étape (passe) du tri à bulles consiste à parcourir tous les éléments du tableau et à permuter l'élément courant ($T[i]$) avec l'élément suivant ($T[i+1]$) s'ils sont en désordre.
- On dit qu'on a une permutation s'il existe (i,j) tels que $i < j$ et $T[i] > T[j]$ (*Pour un tri croissant*).
- Un tableau est trié s'il n'y a aucune permutation.

4. Tri d'un tableau – Tri à bulles

Exemple :

Soit les éléments du tableau suivant [5,1,3,7,2], voici ce que donne le tri à bulles sur ce tableau :

- Première étape :

[5, 1, 3, 7, 2] → [1, 5, 3, 7, 2] → [1, 3, 5, 7, 2] → [1, 3, 5, 7, 2] → [1, 3, 5, 2, 7]

- Deuxième étape :

[1, 3, 5, 2, 7] → [1, 3, 5, 2, 7] → [1, 3, 5, 2, 7] → [1, 3, 2, 5, 7]

- Troisième étape :

[1, 3, 2, 5, 7] → [1, 3, 2, 5, 7] → [1, 2, 3, 5, 7]

- Quatrième étape :

[1, 2, 3, 5, 7] → [1, 2, 3, 5, 7]

4. Tri d'un tableau – Tri à bulles

Algorithme (Pseudo-code) :

```
...
Pour i de N – 1 à 1 Pas -1 faire    //N est le nombre d'éléments du tableau T
    Pour j de 0 à i – 1 faire
        Si T[j] > T[j + 1] alors
            temp ← T[j];
            T[j] ← T[j+1];
            T[j+1] ← temp;
        FinSi
    FinPour
FinPour
...
```

5. Recherche d'un élément dans un tableau

Pour effectuer la recherche d'un élément dans un tableau, deux méthodes de recherche sont considérées selon que le tableau est trié ou non :

- La **recherche séquentielle** pour un **tableau non trié**;
- La **recherche dichotomique** pour un **tableau trié**;

5. Recherche d'un élément dans un tableau – Recherche séquentielle

- La recherche séquentielle consiste à parcourir un tableau **non trié** à partir du début et s'arrêter dès qu'une première occurrence de l'élément sera trouvée.
- Le tableau sera parcouru du début à la fin si l'élément n'y figure pas.

5. Recherche d'un élément dans un tableau – Recherche séquentielle

Recherche de la valeur x dans un tableau T de N éléments :

```
...
i ← 0;
Existe ← Faux;
TantQue (i < N) ET (Existe = Faux) faire
    Si (T[i] = x) alors                                // x est l'élément recherché
        Existe ← Vrai;
    Sinon
        i ← i+1;
    FinSi
FinTantQue
Si Existe alors                                        // c'est équivalent à écrire Si Existe=Vrai alors
    Ecrire (x, " est situé dans la ", i+1, "eme position du tableau ");
Sinon
    Ecrire (x, " n'appartient pas au tableau");
FinSi
```

5. Recherche d'un élément dans un tableau – Recherche dichotomique

- Dans le cas où le **tableau est ordonné**, on peut améliorer l'efficacité de la recherche en utilisant la méthode de **recherche dichotomique**.

Principe :

- Diviser par 2 le nombre d'éléments dans lesquels on cherche la valeur x à chaque étape de la recherche. Pour cela on compare x avec $T[\text{milieu}]$:
 - Si $x < T[\text{milieu}]$, il suffit de chercher x dans la 1^{ère} moitié du tableau entre ($T[0]$ et $T[\text{milieu}-1]$);
 - Si $x > T[\text{milieu}]$, il suffit de chercher x dans la 2^{ème} moitié du tableau entre ($T[\text{milieu}+1]$ et $T[N-1]$);

5. Recherche d'un élément dans un tableau – Recherche dichotomique

Recherche de la valeur x dans un tableau T trié de N éléments :

```
...
inf ← 0; sup ← N-1; Existe ← Faux;
TantQue (inf ≤ sup) ET (Existe = Faux) faire
    milieu ← (inf+sup) div 2;
    Si (x=T[milieu]) alors
        Existe ← Vrai;
    SinonSi (x>T[milieu]) alors
        inf ← milieu+1;
    Sinon
        sup ← milieu-1;
    FinSi
FinTantQue
Si Existe alors    Ecrire (x, " appartient au tableau");
Sinon             Ecrire (x, " n'appartient pas au tableau");
FinSi
```